Chương 5

ỨNG DỤNG HỌC SÂU

Chương 5 sẽ tập trung vào hai bài toán: bài toán phân lớp chó mèo và bài toán phân loại tin tức, cùng với các mô hình học sâu đã trình bày ở các chương trên. Phần đầu trình bày bài toán phân loại chó mèo, chi tiết các bước từ chuẩn bị dữ liệu, xây dựng mô hình phân loại, huấn luyện đến đánh giá và kiểm thử mô hình. Bài toán tiếp theo là bài toán phân loại tin tức, bao gồm các giai đoạn chuẩn bị dữ liệu, thăm dò và thống kê đặc trưng của dữ liệu, tiền xử lý dữ liệu, biểu diễn văn bản sang vector đặc trưng, chia tập dữ liệu thành tập huấn luyện và kiểm thử, sử dụng mô hình học sâu phổ biến như CNN, RNN và LSTM được triển khai với bài toán phân loại tin tức. Qua hai bài toán này, giúp sinh viên có cái nhìn tổng quan khi áp dụng cùng một bài toán với các mô hình khác nhau như thế nào.

5.1 Bài toán phân loại chó mèo

Phân loại hình ảnh chó và mèo là một bài toán tự động nhận diện cho hình ảnh đầu vào là "chó"hoặc "mèo"dựa trên các đặc trưng trực quan trong ảnh. Mỗi hình ảnh sẽ được xác định thuộc một trong hai lớp, giúp hệ thống hiểu được nội dung hình ảnh một cách chính xác. Việc phân loại này không chỉ hỗ trợ các ứng dụng thực tế mà còn là một bài toán nền tảng trong thị giác máy tính và học sâu.

- Đầu vào: Hình ảnh chứa một con vật (chó hoặc mèo).
- Đầu ra: Nhãn phân loại cho biết ảnh thuộc loại "chó"hay "mèo".

Phân loại hình ảnh chó và mèo trong thực tế gặp phải nhiều thách thức đặc thù:

- Đặc điểm hình ảnh phức tạp: Chó và mèo có nhiều giống loài khác nhau với sự đa dạng về màu sắc, kích thước, hình dạng và tư thế.
- Bối cảnh ảnh thay đổi: Hình ảnh có thể được chụp trong nhiều điều kiện ánh sáng, góc nhìn, và phông nền khác nhau, gây khó khăn cho mô hình trong việc trích xuất đặc trưng ổn định.
- Sự tương đồng hình ảnh: Một số đặc điểm của chó và mèo có thể rất giống nhau (đặc biệt là khi chúng còn nhỏ), đòi hỏi mô hình phải đủ tinh vi để phân biệt.

Bài toán phân loại chó và mèo có những ý nghĩa quan trọng như sau:

- Phát triển và kiểm thử mô hình AI: Đây là một bài toán kinh điển được dùng rộng rãi để huấn luyện và đánh giá các mô hình học sâu trong thị giác máy tính.
- Tăng cường khả năng nhận diện đối tượng: Ứng dụng trong các hệ thống camera giám sát, trợ lý ảo, và robot nhận diện vật nuôi trong nhà.
- Nâng cao trải nghiệm người dùng: Trong các nền tảng chia sẻ hình ảnh hoặc thương mại điện tử, việc tự động phân loại hình ảnh giúp cải thiện khả năng tìm kiếm và gợi ý nội dung.
- Tự động hóa quá trình quản lý dữ liệu hình ảnh: Hỗ trợ hệ thống trong việc tổ chức, lưu trữ và phân tích dữ liệu hình ảnh hiệu quả hơn.

5.1.1 Chuẩn bị dữ liệu

Trong bài này chúng tôi sử dụng tập dữ liệu trên trang Kaggle của tác giả Kunal Gupta. Cách download tập dữ liệu như dưới đây:

```
import opendatasets as opd
opd.download("https://www.kaggle.com/datasets/kunalgupta2616/dog-vs-cat-images-data")
```

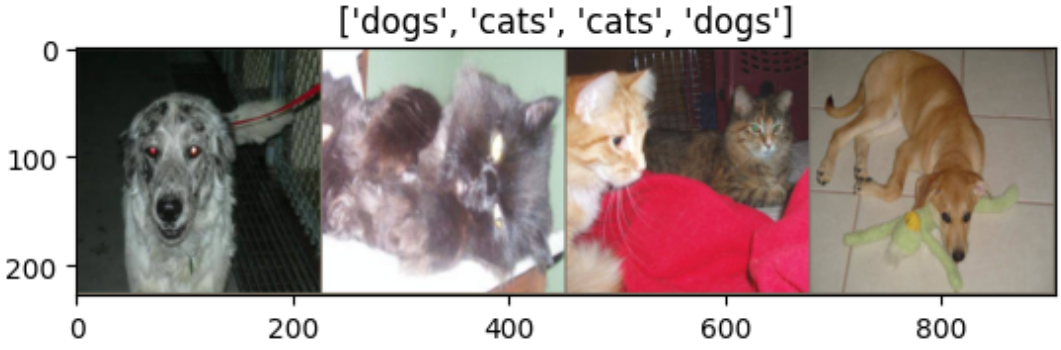
Tiền xử lý dữ liệu và load data

```
import torch
from torch import nn, optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
from torchvision import models
import matplotlib.pyplot as plt
import numpy as np

transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])
train_data = datasets.ImageFolder(
'/content/dog-vs-cat-images-data/dogcat/train', transform=transform)
val_data = datasets.ImageFolder(
'/content/dog-vs-cat-images-data/dogcat/validation', transform=transform)
train_loader = DataLoader(train_data, batch_size=32, shuffle=True)
val_loader = DataLoader(val_data, batch_size=32, shuffle=True)
```

Visualize tập dữ liệu

```
# Display random 4 image on test dataset
def imshow(inp, title=None):
    inp = inp.numpy().transpose((1, 2, 0))
    mean = np.array([0.485, 0.456, 0.406])
    std = np.array([0.229, 0.224, 0.225])
    inp = std * inp + mean
    inp = np.clip(inp, 0, 1)
    plt.imshow(inp)
    if title is not None:
        plt.title(title)
    plt.pause(0.001)
inputs, classes = next(iter(train_loader))
out = torchvision.utils.make_grid(inputs[:4])
imshow(out, title=[train_data.classes[x] for x in classes[:4]])
```



Hình 5.1: Visualize tập train

Tổng quan về tập dữ liệu

5.1.2 Xây dựng mô hình phân loại

Kiểm tra GPU có hoạt động không

```
device = "cuda" if torch.cuda.is_available() else "cpu"
device
```

Dataset	Train	Validation	Test
Cat and Dog	25k	8k	8k

Bảng 5.1: Bảng thống kê tập dữ liệu

Xây dựng mô hình

```
model = models.vgg16(pretrained=True)
for param in model.parameters():
    param.requires_grad = False
model.classifier[6] = nn.Linear(4096, 2)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
```

5.1.3 Huấn luyện mô hình

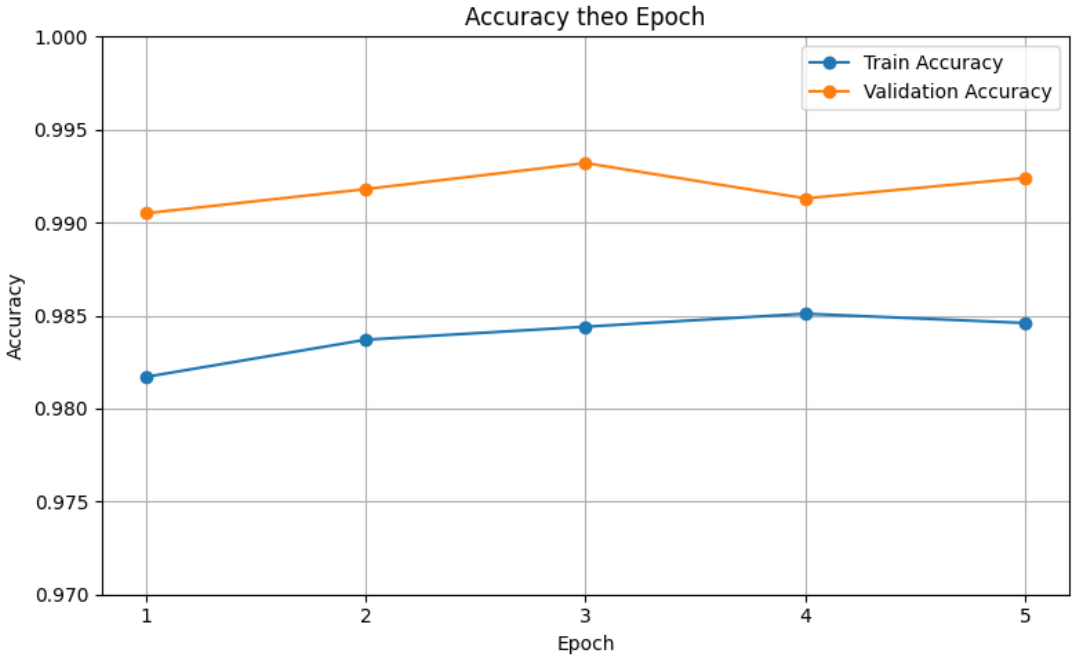
Huấn luyện mô hình phân loại chó mèo

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.classifier.parameters(), lr=0.001)
def train_model(model, train_loader, val_loader, criterion, optimizer, num_epochs=5):
    best_model_wts = model.state_dict()
    best_acc = 0.0
    for epoch in range(num_epochs):
        print(f'Epoch {epoch + 1}/{num_epochs}')
        print('-' * 10)
        # Training
        model.train()
        running_loss = 0.0
        running_corrects = 0
        for inputs, labels in train_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            # Zero the parameter gradients
            optimizer.zero_grad()
            # Forward + backward + optimize
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
            # Statistics
            _, preds = torch.max(outputs, 1)
            running_loss += loss.item() * inputs.size(0)
            running_corrects += torch.sum(preds == labels.data)
        epoch_loss = running_loss / len(train_loader.dataset)
        epoch_acc = running_corrects.double() / len(train_loader.dataset)
        print(f'Training Loss: {epoch_loss:.4f} Acc: {epoch_acc:.4f}')
        # Validation
        model.eval()
        val_corrects = 0
        for inputs, labels in val_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            with torch.no_grad():
                outputs = model(inputs)
            _, preds = torch.max(outputs, 1)
            val_corrects += torch.sum(preds == labels.data)

        val_acc = val_corrects.double() / len(val_loader.dataset)
        print(f'Validation Accuracy: {val_acc:.4f}')
        # Save the best model
        if val_acc > best_acc:
            best_acc = val_acc
            best_model_wts = model.state_dict()
    print(f'Best Validation Accuracy: {best_acc:.4f}')
    model.load_state_dict(best_model_wts)
    return model
model = train_model(model, train_loader, val_loader, criterion, optimizer, num_epochs=5)
```

Quá trình train mô hình

Quá trình huấn luyện cho thấy mô hình liên tục cải thiện qua từng epoch. Độ chính xác trên tập train tăng đều, đạt gần 98.5%, trong khi độ chính xác trên tập validation duy trì ở mức rất cao, dao động quanh 99%



Hình 5.2: Biểu đồ accuracy trong 5 epochs

5.1.4 Đánh giá và kiểm thử mô hình

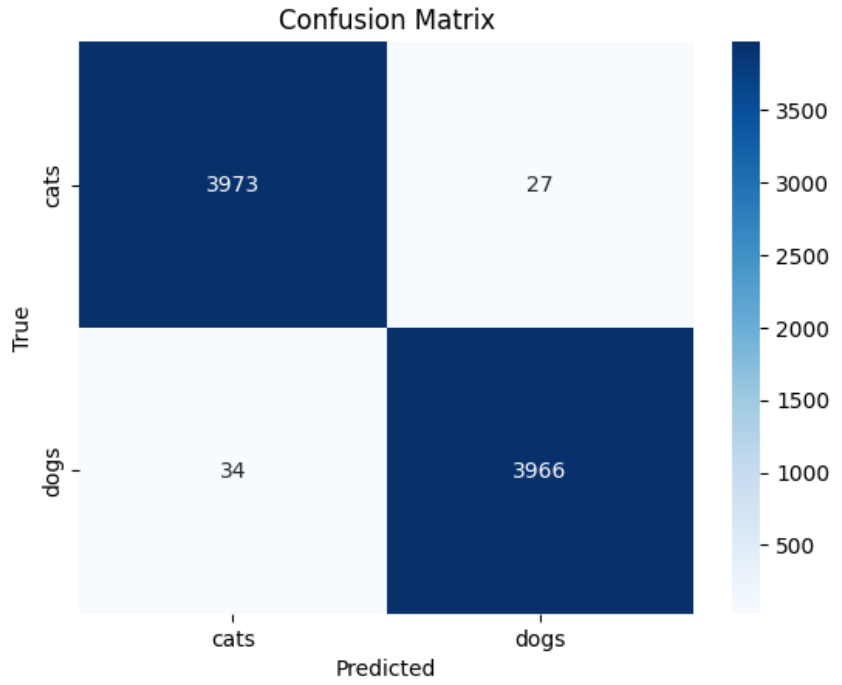
Confusion matrix (Ma trận nhầm lẫn) Thử nghiệm trên tập test

```
test_dir = "/content/dog-vs-cat-images-data/dogcat/test1/test1"
random_images = random.sample(os.listdir(test_dir), 6)
model.eval()
plt.figure(figsize=(12, 8))

for i, img_name in enumerate(random_images):
    img_path = os.path.join(test_dir, img_name)
    image = Image.open(img_path).convert('RGB')
    input_tensor = transform(image).unsqueeze(0).to(device)

    with torch.no_grad():
        output = model(input_tensor)
        _, pred = torch.max(output, 1)
        label = class_names[pred.item()]
    plt.subplot(2, 3, i + 1)
    plt.imshow(image)
    plt.title(f'Predicted: {label}')
    plt.axis('off')

plt.tight_layout()
plt.show()
```



Hình 5.3: Bảng ma trận nhầm lẫn



Hình 5.4: Kết quả thử nghiệm trên tập test

5.2 Bài toán phân loại tin tức

Phân loại tin tức là quá trình tự động gán nhãn cho các bài viết, tin tức dựa trên nội dung của chúng. Mỗi bài viết được phân vào một hoặc nhiều danh mục, chẳng hạn như kinh tế, chính trị, giáo dục, thể thao, hoặc giải trí. Việc phân loại này giúp người dùng truy cập thông tin nhanh chóng và chính xác, đồng thời giúp hệ thống quản lý dữ liệu hiệu quả hơn.

- Đầu vào: Đoạn văn bản báo tiếng Việt.
- Đầu ra: Phân lớp văn bản. Chúng ta đi xác định xem văn bản này thuộc loại nào trong 12 lớp: Chính trị, Xã hội, KHCN, Kinh tế, Thế giới, Văn hóa, Giải trí, Du lịch, Giáo dục, Sức khỏe, Pháp luật, Thể thao.

Phân loại tin tức tiếng Việt đối mặt với nhiều thách thức đặc thù:

Đặc tính tiếng Việt: Tiếng Việt là ngôn ngữ đơn âm tiết, dựa trên hệ chữ quốc ngữ với dấu thanh. Các từ có thể mang nhiều nghĩa trong các ngữ cảnh khác nhau. Ngôn ngữ không chính quy: Tin tức, bài viết thường mang ngôn ngữ biến thể, bao gồm tiếng lóng, từ vay mượn, hoặc sai chính tả.

Bài toán phân loại tin tức có những ý nghĩa quan trọng như sau:

- Tối ưu hóa quy trình xử lý thông tin: Trong bối cảnh lượng thông tin tăng nhanh chóng, phân loại tin tức giúp giảm thiểu thời gian và công sức cần thiết để xác định nội dung mong muốn.
- Nâng cao hiệu quả truy xuất thông tin: Giúp người dùng dễ dàng truy cập vào các danh mục thông tin có liên quan đến nhu cầu của họ.
- Hỗ trợ ra quyết định: Trong nhiều ngành như kinh doanh, chính trị, và giáo dục, việc phân loại tin tức đúng đắn giúp cung cấp dữ liệu có giá trị cho các quyết định chiến lược.
- Tăng cường khả năng tự động hóa: Các hệ thống AI và machine learning được tích hợp có thể phân loại và đề xuất tin tức tự động, từ đó tiết kiệm nguồn lực nhân sự.

5.2.1 Chuẩn bị dữ liệu

Trong bài toán này chúng tôi sử dụng tập dữ liệu craw từ các trang báo điện tử ngôn ngữ tiếng Việt của tác giả Nguyễn Minh Quang[1]. Cấu trúc file dữ liệu dạng json như Hình 5.5


```
"url": "https://vietnamnet.vn/dien-bien-to-chuc-hoi-nghi-tap-huan-cong-tac-nhan-quyen-nam-2023-2205811.html",
"title": "Điện Biên tổ chức Hội nghị tập huấn công tác nhân quyền năm 2023",
"description": "Ngày 4/10, Ban Chỉ đạo Nhân quyền tỉnh Điện Biên phối hợp với Văn phòng Thường trực về Nhân quyền
"content": "Tại Hội nghị, các đại biểu đã được các báo cáo viên của Văn phòng Thường trực về nhân quyền, Ban Tôn
"metadata": {
    "cat": "DÂN TỘC - TÔN GIÁO",
    "subcat": "CHÍNH SÁCH",
    "published_date": 1697239440,
    "author": "Unknown"
},
"web_news": "vietnamnet"
```


Hình 5.5: Cấu trúc file dữ liệu

5.2.2 Thăm dò và thống kê đặc trưng của dữ liệu

Khai báo các thư viện sử dụng

```python
import json
import requests
from pprint import pprint
import re
import string
import random
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm.notebook import tqdm
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.naive_bayes import MultinomialNB
from sklearn.ensemble import RandomForestClassifier
from sklearn.utils import shuffle
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.feature_extraction.text import TfidfVectorizer
from collections import defaultdict
import warnings
import sys
import torch
```

Kiểm tra GPU có khả dụng không

```python
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
if device.type == 'cpu':
    raise RuntimeError("No GPU found")

print(f'Using device: {device}')
```

[1] https://raw.githubusercontent.com/nmquang003/AI4E/main/10k_news.json

Tải file chứa dữ liệu

```
url = 'https://raw.githubusercontent.com/nmquang003/AI4E/main/10k_news.json'

# Tai noi dung file
response = requests.get(url)

# Kiem tra ma phan hoi dam bao phan hoi thanh cong
if response.status_code == 200:
    try:
        data = response.json()  # doc noi dung file

        # Hien thi phan tu dau tien cua file
        if isinstance(data, list) and len(data) > 0:
            pprint(json.dumps(data[0], indent=4, ensure_ascii=False))
        else:
            print("Du lieu json khong phai danh sach hoac danh sach rong.")

    except json.JSONDecodeError as e:
        print("JSONDecodeError: Khong the phan tich noi dung file")
        print("Noi dung khong phai file json hop le")
        print(response.text)  # In noi dung phan hoi
else:
    print(f"yeu cau that bai voi ma {response.status_code}")
    print("Noi dung phan hoi:", response.text)
```

Kiểm tra các đầu mục

```
# Trich xuat danh muc (categories)

categories = [article['metadata']['cat'] for article in data]

# Dem so luong tung danh muc
category_counts = {}
for category in categories:
    if category in category_counts:
        category_counts[category] += 1
    else:
        category_counts[category] = 1

# In ra danh muc va so luong tuong ung
for category, count in category_counts.items():
    print(f"Category: {category}, Count: {count}")
```

Kết quả của đoạn code trong Hình 5.6.


```
"url": "https://vietnamnet.vn/dien-bien-to-chuc-hoi-nghi-tap-huan-cong-tac-nhan-quyen-nam-2023-2205811.html",
"title": "Điện Biên tổ chức Hội nghị tập huấn công tác nhân quyền năm 2023",
"description": "Ngày 4/10, Ban Chỉ đạo Nhân quyền tỉnh Điện Biên phối hợp với Văn phòng Thường trực về Nhân quyền
"content": "Tại Hội nghị, các đại biểu đã được các báo cáo viên của Văn phòng Thường trực về nhân quyền, Ban Tôn
"metadata": {
    "cat": "DÂN TỘC - TÔN GIÁO",
    "subcat": "CHÍNH SÁCH",
    "published_date": 1697239440,
    "author": "Unknown"
},
"web_news": "vietnamnet"
```


Hình 5.6: Kết quả đếm số lượng danh mục

Do các đầu mục trong dữ liệu không thống nhất nên bước tiếp theo chúng ta sẽ gộp lại các danh mục về 12 loại như phần mô tả bài toán.

```
# Tu dien anh xa categories ve 12 loai

category_mapping = {
    "CHINH TRI": "Chinh tri",
    "THOI SU": "Chinh tri",
    "XA HOI": "Xa hoi",
    "DOI SONG": "Xa hoi",
    "AN SINH": "Xa hoi",
    "LAO DONG - VIEC LAM": "Xa hoi",
    "TINH YEU - GIOI TINH": "Xa hoi",
    "KHOA HOC - CONG NGHE": "KHCN",
    "SUC MANH SO": "KHCN",
    "So hoa": "KHCN",
    "CONG NGHIEP HO TRO": "KHCN",
```

```
    "KINH DOANH": "Kinh te",
    "Kinh doanh": "Kinh te",
    "THI TRUONG - TIEU DUNG": "Kinh te",
    "BAT DONG SAN": "Kinh te",
    "Bat dong san": "Kinh te",
    "THE GIOI": "The gioi",
    "The gioi": "The gioi",
    "VAN HOA": "Van hoa",
    "GIAI TRI": "Giai tri",
    "Giai tri": "Giai tri",
    "DU LICH": "Du lich",
    "Du lich": "Du lich",
    "GIAO DUC": "Giao duc",
    "Giao duc": "Giao duc",
    "SUC KHOE": "Suc khoe",
    "Suc khoe": "Suc khoe",
    "Cac benh": "Suc khoe",
    "PHAP LUAT": "Phap luat",
    "Phap luat": "Phap luat",
    "THE THAO": "The thao",
    "The thao": "The thao"
}

# Loc va chuyen doi categories
filtered_data = []
for article in data:
    old_category = article['metadata']['cat']
    if old_category in category_mapping:
        new_category = category_mapping[old_category]
        article['metadata']['cat'] = new_category
        filtered_data.append(article)

# In bai bao sau khi loc va chuyen doi categories
print(f"Filtered articles count: {len(filtered_data)}")

# Trich xuat danh muc (categories)
categories = [article['metadata']['cat'] for article in filtered_data]

# Dem so luong danh muc
category_counts = {}
for category in categories:
    if category in category_counts:
        category_counts[category] += 1
    else:
        category_counts[category] = 1

# Ve bieu do cot
plt.figure(figsize=(10, 6))  # Dieu chinh kich thuoc bieu do
ax = sns.barplot(x=list(category_counts.keys()), y=list(category_counts.values()))
plt.xlabel('Category')
plt.ylabel('Count')
plt.title('Article Count per Category')
plt.xticks(rotation=45)

# Them so luong tren moi cot
for p in ax.patches:
    ax.annotate(f'{int(p.get_height())}', (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='center', xytext=(0, 10), textcoords='offset points')

plt.show()
```

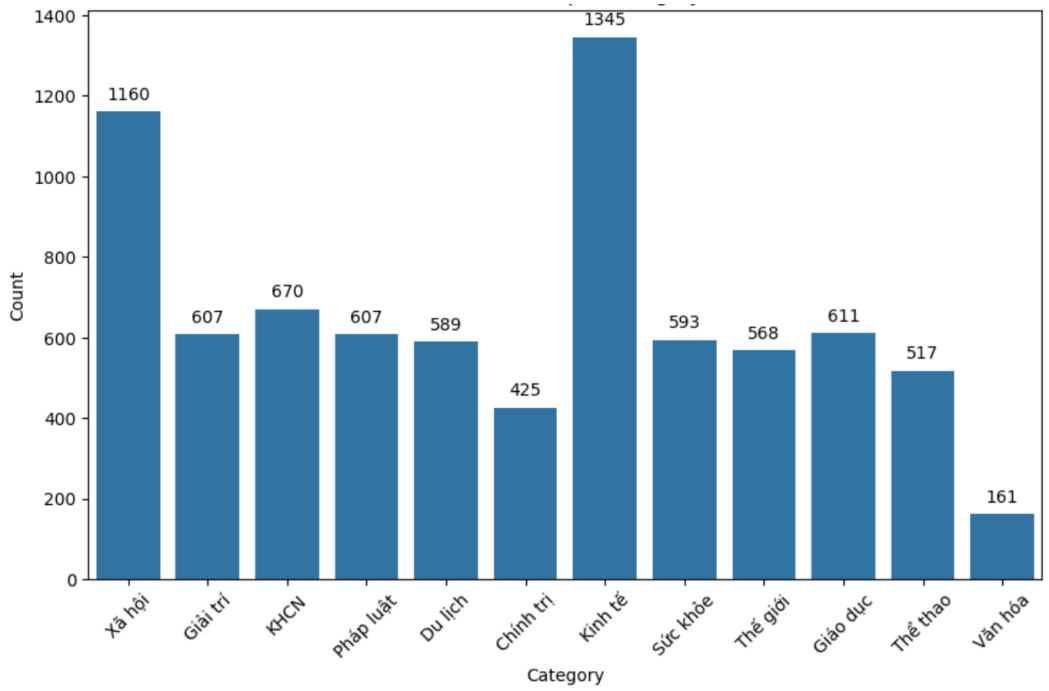
Kết quả hiển thị biểu đồ số lượng bài báo trên mỗi danh mục categories trong Hình 5.7. Hình vẽ cho thấy dữ liệu trên các lớp chưa cân bằng. Do vậy, bước tiếp theo chúng ta thực hiện cân bằng dữ liệu.

Cân bằng dữ liệu trên mỗi lớp

```
category_data = defaultdict(list)
for article in filtered_data:
    category_data[article['metadata']['cat']].append(article)

# Xac dinh so luong toi da tren moi category
number_sample = 4096
max_samples_per_category = number_sample // len(category_data)

# Chon ngau nhien cac mau tren moi category, gioi han boi max_samples_per_category
balanced_data = []
for category, items in category_data.items():
```



Hình 5.7: Số lượng bài báo trên mỗi danh mục

```
    if len(items) > max_samples_per_category:
        balanced_data.extend(random.sample(items, max_samples_per_category))
    else:
        balanced_data.extend(items)

# Neu so luong mau ko du number_sample, bo sung them mau ngau nhien balanced_data
if len(balanced_data) < number_sample:
    additional_samples = random.sample(balanced_data, number_sample - len(balanced_data))
    balanced_data.extend(additional_samples)

filtered_data_to_use = balanced_data[:number_sample if number_sample < len(balanced_data)
else len(balanced_data)]
print(f"Filtered articles to use count: {len(filtered_data_to_use)}")

# Dem so luong tung danh muc
category_counts = defaultdict(int)
for item in filtered_data_to_use:
    category_counts[item['metadata']['cat']] += 1

# Ve bieu do cot
plt.figure(figsize=(10, 6))  # Dieu chinh kich thuoc bieu do
ax = sns.barplot(x=list(category_counts.keys()), y=list(category_counts.values()))
plt.xlabel('Category')
plt.ylabel('Count')
plt.title('Article Count per Category')
plt.xticks(rotation=45)

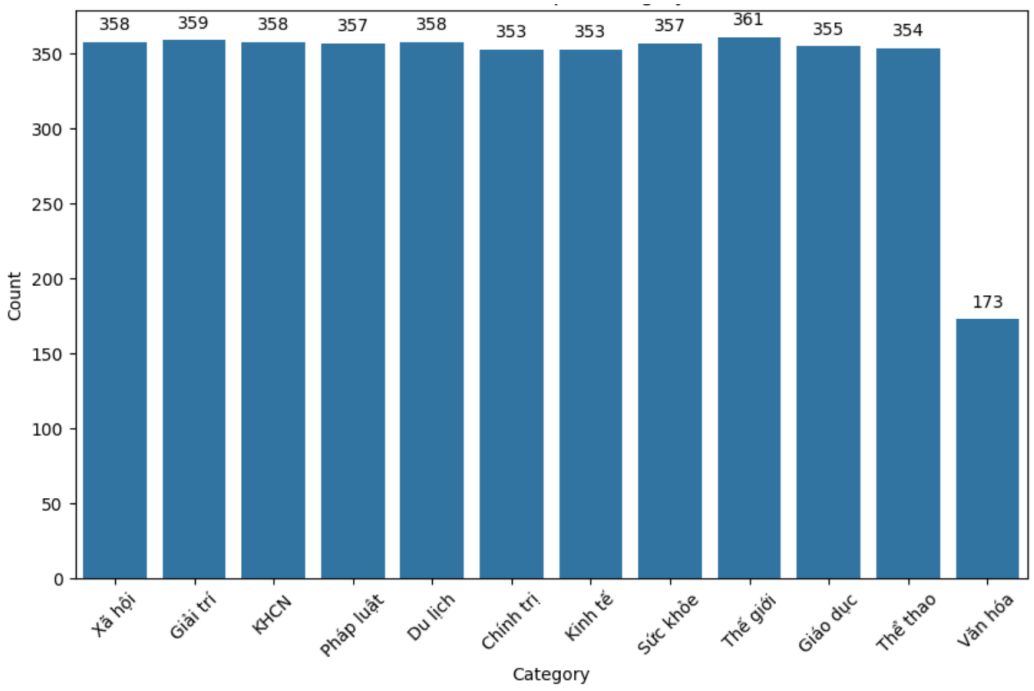
# Them so luong tren moi cot
for p in ax.patches:
    ax.annotate(f'{int(p.get_height())}', (p.get_x()
    + p.get_width() / 2., p.get_height()),
    ha='center', va='center', xytext=(0, 10), textcoords='offset points')

plt.show()
```

Hình 5.8 mô tả dữ liệu sau khi cân bằng.

5.2.3 Tiền xử lý dữ liệu

Tiền xử lý dữ liệu nói chung và trên văn bản nói riêng là một bước quan trọng và không thể thiếu trong các bài toán xử lý ngôn ngữ tự nhiên (Natural Language Processing - NLP). Mục tiêu của quá trình này là biến đổi dữ liệu văn bản thô thành một dạng có cấu trúc, giúp các mô hình học máy hoặc học sâu dễ



Hình 5.8: Số lượng bài báo trên mỗi nhãn sau khi cân bằng

dàng tiếp nhận và xử lý. Quá trình tiền xử lý bao gồm nhiều bước cơ bản nhưng đóng vai trò quyết định đến hiệu suất và độ chính xác của mô hình. Sau đây là một số thao tác trong tiền xử lý văn bản:

- Chuẩn hóa văn bản: bao gồm việc chuyển đổi toàn bộ văn bản về chữ thường nhằm giảm thiểu sự khác biệt không cần thiết giữa các từ viết hoa và viết thường. Tiếp theo, văn bản được phân tách thành các đơn vị nhỏ hơn gọi là token thông qua kỹ thuật tokenization. Tokenization giúp chia văn bản thành các từ, cụm từ, hoặc ký hiệu riêng lẻ, tạo điều kiện thuận lợi cho việc xử lý tiếp theo.
- Loại bỏ từ dừng (stop words), bao gồm các từ phổ biến như "the", "is", "and", nhưng ít đóng góp ý nghĩa trong việc phân tích ngữ nghĩa. Quá trình này giúp giảm kích thước dữ liệu và tập trung vào các từ quan trọng hơn. Ngoài ra, các kỹ thuật khác như stemming và lemmatization thường được áp dụng để chuẩn hóa các từ về dạng gốc. Stemming thực hiện cắt gọn các từ về gốc cơ bản của chúng, trong khi lemmatization sử dụng từ điển ngữ pháp để trả về dạng từ có ý nghĩa hơn.
- Các thao tác khác như loại bỏ các giá trị trống và xử lý các văn bản bị lỗi cũng rất cần thiết để đảm bảo tính toàn vẹn của dữ liệu. Bên cạnh đó, việc chuyển đổi các ký tự đặc biệt hoặc từ không phải tiếng Anh thành dạng tương ứng cũng là một phần quan trọng trong các bài toán NLP đa ngôn ngữ. Đối với các dữ liệu đặc thù như đánh giá sản phẩm hoặc bình luận trên mạng xã hội, xử lý biểu tượng cảm xúc (emoji) hoặc viết tắt có thể cải thiện đáng kể độ chính xác của mô hình.

Nhìn chung, tiền xử lý dữ liệu văn bản không chỉ cải thiện chất lượng đầu vào cho mô hình mà còn giảm thiểu chi phí tính toán và cải thiện hiệu quả học tập. Sự lựa chọn và thứ tự các bước tiền xử lý phụ thuộc vào bài toán cụ thể, loại mô hình được sử dụng và ngôn ngữ xử lý. Do đó, đây là một phần quan trọng cần được thực hiện một cách cẩn thận và phù hợp trong bất kỳ bài toán NLP nào.

Tiền xử lý văn bản không được trình bày kỹ ở cuốn sách này. Trong bài tập minh họa của cuốn sách này, chúng tôi trình bày một số bước cơ bản dưới đây:

Làm sạch dữ liệu

```
# Danh sach stop word tieng Viet
stop_words = set([
    "bi", "boi", "ca", "cac", "cai", "can", "cang", "chi", "chiec", "cho", "chu", "chua",
    "chuyen", "co", "co_the", "cu", "cua", "cung", "cung", "da", "dang", "day", "de",
    "den_noi", "deu", "dieu", "do", "do", "duoc", "duoi", "gi", "khi", "khong",
    "la", "lai", "len", "luc", "ma", "moi", "nay", "nen", "neu", "ngay",
    "nhieu", "nhu", "nhung", "nhung", "noi", "nua", "phai", "qua", "ra", "rang",
    "rang", "rat", "roi", "sau", "se", "so", "su", "tai", "theo", "thi", "tren",
    "truoc", "tu", "tung", "va", "van", "vao", "vay", "vi", "viec", "voi", "vua"
])

```

```
# Trich xuat du lieu tu cac bai bao da loc
texts = []
labels = []

# Lap qua du lieu tung bai bao da loc
for article in filtered_data_to_use:
    # ket hop tieu de va noi dung vao text
    text = article.get('title', '') + ' ' + article.get('content', '')
    # Lay nhan (category) cua bai bao
    label = article['metadata']['cat']
    # Them van ban va nhan vao danh sach
    texts.append(text)
    labels.append(label)

# Ham tien xu ly van ban
def preprocess_text(text):
    # Chuyen van ban thanh chu thuong
    text = text.lower()
    # Loai  bo dau cau
    text = re.sub(f"[{re.escape(string.punctuation)}]", "", text)
    # Loai bo khoang trang thua
    text = re.sub(r'\s+', ' ', text).strip()
    # Tach van ban thanh cac tu (tokens)
    tokens = text.split()
    # Dem tong so tu ban dau
    total_tokens = len(tokens)
    # Loai bo cac tu trong stop word
    tokens = [token for token in tokens if token not in stop_words]
    # Tinh % cac stop word bi loai bo
    removed_stop_words_percent = (total_tokens - len(tokens)) / total_tokens * 100
    # Tra ve cac  van ban da dc xu ly va phan tram stop word bi loai bo
    return ' '.join(tokens), removed_stop_words_percent

# Danh sach van ban da qua tien xu ly
processed_texts = []
# Danh sach chua phan tram cac tu stop word da bi loai bo trong tung van ban
sum_stop_words_percents = 0

# su dung tqdm hien thi tien trinh xu ly van ban
tqdm_iter = tqdm(texts, desc="Processing texts")

# Lap qua tung van ban trong van ban goc
for i, text in enumerate(tqdm_iter):
    # Tien xu ly van ban va tinh phan tram tu stop word da bi loai bo
    processed_text, stop_words_percent = preprocess_text(text)
    # Them van ban da qua tien xu ly vao danh sach
    processed_texts.append(processed_text)
    # Them phan tram tu stop word da bi loai bo vao danh sach
    sum_stop_words_percents += stop_words_percent
    # Tinh % trung binh cua stop word da bi loai bo
    avg_stop_words_percent = sum_stop_words_percents / (i+1)
    # cap nhat thanh tien trinh
    tqdm_iter.set_postfix({"avg_stop_words_percent": f"{avg_stop_words_percent:.2f}%"})

# Cap nhat danh sach van ban goc voi van ban da qua tien xu ly
texts = processed_texts
# Tinh % trung bi cuoi cung cua cac tu stop words da bi loai bo
average_stop_words_percent = sum_stop_words_percents / len(texts)

# In % trung binh cac tu stop words da bi loai bo
print(f"Average percentage of stop words removed: {average_stop_words_percent:.2f}%")
```

Kết quả sau bước này:

```
Processing texts:100% 4096/4096 [00:06<00:00, 851.07it/s, avg_stop_words_percent=18.67%]

Average percentage of stop words removed: 18.67%
```

Test thử văn bản sau xử lý

```
print("van ban sau xu ly: ", texts[0])
print("Nhan van ban: ", labels[0])
```

5.2.4 Biểu diễn văn bản sang véc tơ đặc trưng

Bước này chúng ta thực hiện chuyển đổi từ hoặc văn bản sang véc tơ. Véc tơ này gọi là véc tơ đặc trưng. Véc tơ này càng mang nhiều ngữ nghĩa thì các mô hình học càng tốt. Có nhiều phương pháp biến đổi từ sang một véc tơ như véc tơ one-hot, sử dụng if.idf; word embedding (Word2vec, Glove, Bert). Trong phần này chúng ta cũng không tìm hiểu chi tiết các phương pháp này. Phần này sẽ được trình bày rõ hơn trong tài liệu về xử lý ngôn ngữ tự nhiên. Trong giới hạn của tài liệu này, chúng ta hiểu được các bước thực hiện và khai thác sử dụng thư viện có sẵn. Trong xử lý ngôn ngữ tự nhiên, biểu diễn văn bản thành véc tơ đặc trưng là bước cần thiết để các thuật toán học máy hoặc học sâu có thể hiểu và xử lý dữ liệu văn bản. Có nhiều thư viện hỗ trợ biểu diễn văn bản sang véc tơ đặc trưng như sklearn.feature_extraction.text, gensim.models, spacy, Hugging Face Transformers, framework học sâu như TensorFlow và PyTorch hỗ trợ việc biểu diễn văn bản bằng cách sử dụng các lớp nhúng từ như Embedding.

Do bài toán phân loại tin tức làm việc với dữ liệu tiếng Việt cho nên chúng tôi sử dụng mô hình PhoBERT[2] để chuyển word thành véc tơ. Mô hình PhoBERT là một biến thể của BERT (Bidirectional Encoder Representations from Transformers) được tối ưu hóa riêng cho tiếng Việt. PhoBERT chuyển đổi từ ngữ hoặc câu văn tiếng Việt thành các vector nhúng (word embeddings), từ đó mô hình hóa các mối quan hệ ngữ nghĩa và ngữ cảnh giữa các từ. Đây là một trong những công cụ mạnh mẽ nhất hiện nay cho xử lý ngôn ngữ tự nhiên (NLP) trong tiếng Việt.

```
from torch import nn
from torch.utils.data import Dataset, DataLoader
from transformers import AutoTokenizer, AutoModel
```

Tải PhoBERT Tokenizer và Model đã được huấn luyện trước trên tập dữ liệu tiếng Việt.

```
tokenizer = AutoTokenizer.from_pretrained('vinai/phobert-base')
embeder = AutoModel.from_pretrained('vinai/phobert-base')

# Chuyen mo hinh sang GPU neu co
embeder.to(device)
```

Chuyển word thành véc tơ:

```
def encode_texts(texts, tokenizer, embeder, max_length=256):
    encoded_texts = []  # Danh sach de luu van ban sau khi ma hoa

    for text in tqdm(texts, desc="Chuyen tu thanh vecto"):
        # Chuyen van ban thanh cac tensor dau vao cho mo hinh, cat hoac
        padding cho bang do dai toi da.
        inputs = tokenizer(text, return_tensors="pt", max_length=max_length,
        truncation=True,
        padding="max_length").to(device)

        # Khong tinh gra dien vi chi thuc hien inference.
        with torch.no_grad():
            outputs = embeder(**inputs)
            #Truyen cac tensor dau vao qua mo hinh de lay cac vecto ma hoa.

        # lay vecto ma hoa tu 'last_hidden_state' va loai bo chieu batch.
        encoded_texts.append(outputs.last_hidden_state.squeeze(0))

    # Chuyen danh sach cac tensor thanh tensor duy nhat.
    return torch.stack(encoded_texts)
```

```
test = ["Xin chao, day la cuon hoc sau"]
encode_texts(test, tokenizer, embeder).shape
```

Kết quả sau khi chạy code:

Chuyển từ thành vector: 100% 2/2[00:01<00:00,1.93s/it]

torch.Size([2, 256, 768])

5.2.5 Chia tập dữ liệu thành hai phần Train và Test

```
# Khai bao mang label y
y = np.array(labels)

# Chia tap du lieu thanh tap huan luyen va tap kiem tra
texts_train, texts_test, y_train, y_test = train_test_split(texts, y, test_size=0.2,
```

[2]https://huggingface.co/vinai/phobert-base

```
stratify=y, random_state=42)

# Encode labels
le = LabelEncoder()
y_train_enc = le.fit_transform(y_train)
y_test_enc = le.transform(y_test)

print("Nhan goc: ", y_train[:5])
print("Nhan sau khi ma hoa: ", y_train_enc[:5])

# Shuffle the training data
texts_train, y_train_enc = shuffle(texts_train, y_train_enc, random_state=42)
```

Chuyển tập train và test sang véc tơ sử dụng PhoBERT

```
X_train_phobert = encode_texts(texts_train, tokenizer, embeder)
X_test_phobert = encode_texts(texts_test, tokenizer, embeder)
```

Định nghĩa lớp TextDataset và tạo các đối tượng DataLoader để chuẩn bị dữ liệu cho mô hình học máy hoặc học sâu.

```
# Khai bao lop dung Dataset
class TextDataset(Dataset):
    def __init__(self, texts, labels):
        self.texts = texts
        self.labels = labels

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        return self.texts[idx], self.labels[idx]

# Tao Dataset cho tap huan luyen  va tap kiem thu
train_dataset = TextDataset(X_train_phobert, y_train_enc)
test_dataset = TextDataset(X_test_phobert, y_test_enc)

#Tao DataLoader cho tap huan luyen va tap kiem thu

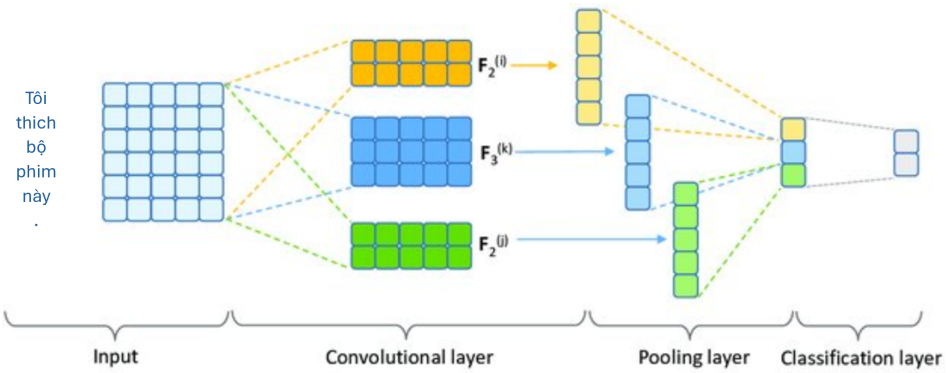
train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=16, shuffle=False)
```

5.2.6 Định nghĩa các mô hình học sâu

Mô hình mạng CNN

CNN cho bài toán phân lớp văn bản thực hiện như sau:

- Ghép các word embedding với nhau để tạo thành ma trận embedding
- Coi ma trận embedding là 1 ảnh.
- Áp dụng CNN lên ma trận embedding trên (Hình 5.9)



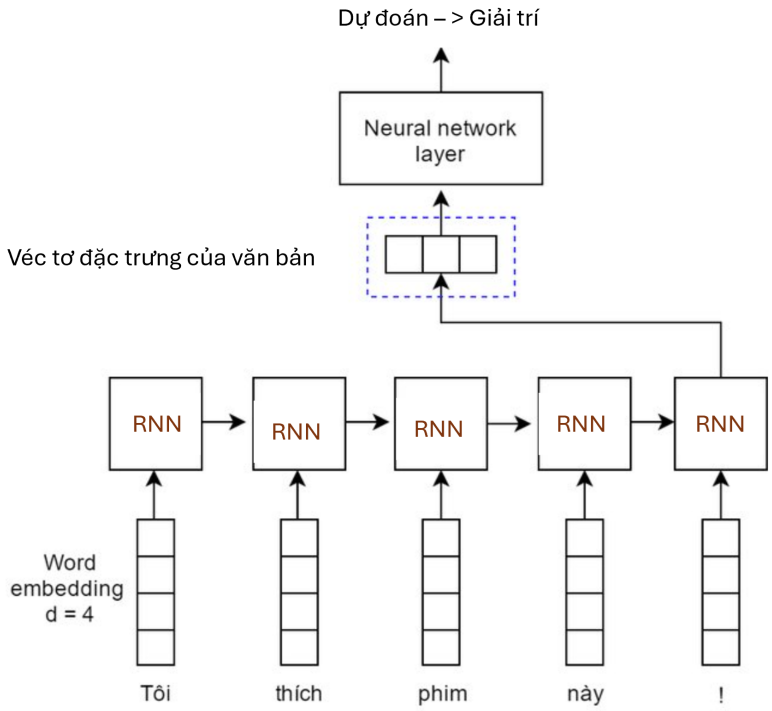
Hình 5.9: Mô hình CNN áp dụng cho bài toán phân loại văn bản

```python
# Xay dung mo hinh CNN
class CNNClassifier(nn.Module):
    def __init__(self, input_size, output_size, num_filters, kernel_size, max_length=256):
        super(CNNClassifier, self).__init__()
        self.conv1 = nn.Conv1d(in_channels=input_size, out_channels=num_filters,
        kernel_size=kernel_size)
        self.pool = nn.MaxPool1d(kernel_size=max_length - kernel_size + 1)
        self.fc = nn.Linear(num_filters, output_size)

    def forward(self, x):
        x = x.permute(0, 2, 1)
        # Doi thu tu de phu hop voi Conv1d (batch_size, input_size, max_length)
        x = self.conv1(x)
        x = self.pool(x).squeeze(2)
        x = self.fc(x)
        return x
```

Mô hình RNN

Hình 5.10 mô tả các bước áp dụng mô hình RNN cho bài toán phân loại tin tức.Chúng ta sử dụng véc tơ hidden-layer ở bước cuối cùng để làm véc tơ đặc trưng của cả văn bản. Véc tơ đặc trưng này đi qua tầng mạng truyền thẳng biến đổi về không gian có số chiều bằng với số nhãn đầu ra của bài toán.



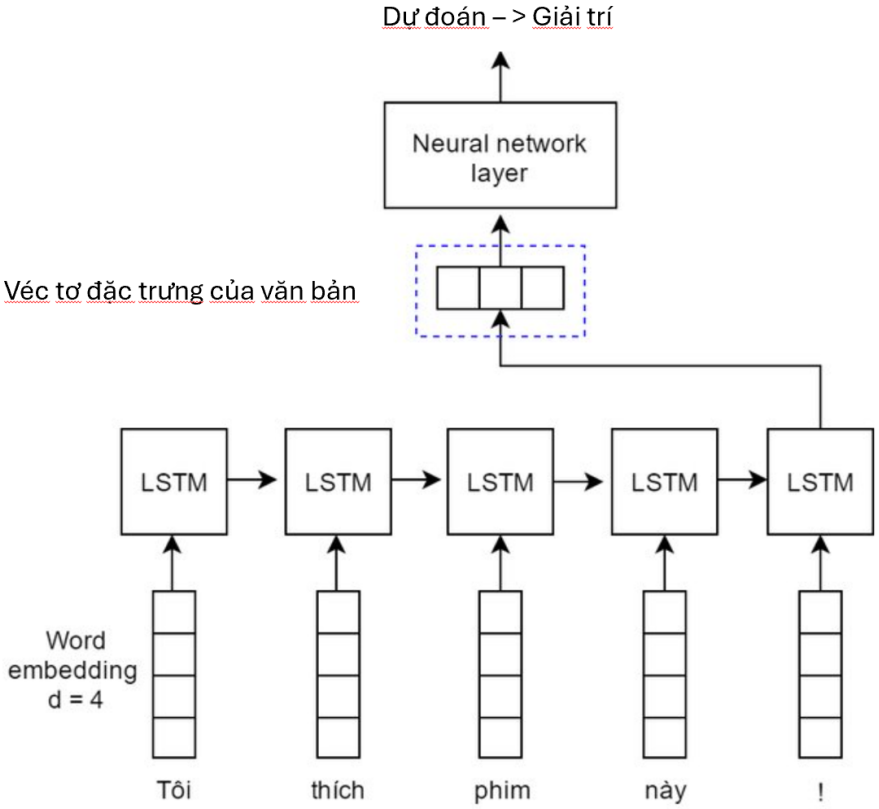
Hình 5.10: Các bước thực hiện bài toán phân loại tin tức sử dụng mô hình RNN

```python
# xay dung mo hinh RNN
class RNNClassifier(nn.Module):
    def __init__(self, input_size, hidden_size, output_size, num_layers=1):
        super(RNNClassifier, self).__init__()
        self.rnn = nn.RNN(input_size, hidden_size, num_layers, batch_first=True)
        self.fc = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        h0 = torch.zeros(1, x.size(0), hidden_size).to(x.device)
        out, _ = self.rnn(x, h0)
        out = self.fc(out[:, -1, :])
        return out
```

Mô hình LSTM

Hình 5.11 mô tả các bước áp dụng mô hình LSTM cho bài toán phân loại tin tức: Tương tự như mô hình RNN, chúng ta sử dụng véc tơ hidden-layer ở bước cuối cùng để làm véc tơ đặc trưng của cả văn bản. Véc tơ đặc trưng này đi qua tầng mạng truyền thẳng biến đổi về không gian có số chiều bằng với số nhãn đầu ra của bài toán.



Hình 5.11: Các bước thực hiện bài toán phân loại tin tức sử dụng mô hình LSTM

```
# Xay dung mo hinh LSTM
class LSTMClassifier(nn.Module):
    def __init__(self, input_size, hidden_size, output_size, num_layers=1):
        super(LSTMClassifier, self).__init__()
        self.lstm = nn.LSTM(input_size, hidden_size, num_layers, batch_first=True)
        self.fc = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        h0 = torch.zeros(1, x.size(0), hidden_size).to(x.device)
        c0 = torch.zeros(1, x.size(0), hidden_size).to(x.device)
        out, _ = self.lstm(x, (h0, c0))
        out = self.fc(out[:, -1, :])
        return out
```

5.2.7 Huấn luyện mô hình học sâu

Thiết lập các thông số

```
input_size = 768
# Kich thuoc vec to an cua PhoBERT
output_size = len(le.classes_)
# so luong lop

hidden_size = 100
num_filters = 10
kernel_size = 3
```

Khởi tạo các mô hình

```
deep_models = {
    'RNN': RNNClassifier(input_size, hidden_size, output_size, num_layers=1).to(device),
    'LSTM': LSTMClassifier(input_size, hidden_size, output_size, num_layers=1).to(device),
    'CNN': CNNClassifier(input_size, output_size, num_filters, kernel_size,
    max_length=256).to(device)
}
```

Hàm huấn luyện và đánh giá mô hình

```
def train_and_evaluate(model, train_loader, test_loader, num_epochs=10, criterion=None,
optimizer_fn=None, scheduler_fn=None):
    # Khoi tao optimize
    optimizer = optimizer_fn(model.parameters())

    # Khoi tao scheduler
    scheduler = scheduler_fn(optimizer)

    for epoch in range(num_epochs):
        model.train()
        for texts, labels in train_loader:
            texts, labels = texts.to(device), labels.to(device)
            outputs = model(texts)
            loss = criterion(outputs, labels)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

        if scheduler is not None:
            scheduler.step()

        print(f"Epoch [{epoch+1}/{num_epochs}], Loss: {loss.item():.4f}")

    model.eval()
    all_preds = []
    all_labels = []
    with torch.no_grad():
        for texts, labels in test_loader:
            texts, labels = texts.to(device), labels.to(device)
            outputs = model(texts)
            _, preds = torch.max(outputs, 1)
            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())

    return all_labels, all_preds
```

Lựa chọn criterion, optimizer, và scheduler

```
# khai bao criterion
criterion = nn.CrossEntropyLoss()
# Ham tao optimizer
def optimizer_fn(params):
    return torch.optim.Adam(params, lr=0.001)
# Ham tao scheduler
def scheduler_fn(optimizer):
    return torch.optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.1)
# So luong epochs
num_epochs = 10
```

Hình vẽ kết quả của mô hình sau khi train

```

fig, axs = plt.subplots(2, 2, figsize=(16, 16))
fig_cm, axs_cm = plt.subplots(2, 2, figsize=(16, 16))

# Luu tru F1 scores cua mo hinh
model_f1_scores = {}

for i, (name, model) in enumerate(deep_models.items()):
    print(f"Training {name} model...")
    y_test, y_pred = train_and_evaluate(model, train_loader, test_loader, num_epochs=10,
                                        criterion=criterion, optimizer_fn=optimizer_fn,
                                        scheduler_fn=scheduler_fn)
    y_test = le.inverse_transform(y_test)
    y_pred = le.inverse_transform(y_pred)

    # Tao bao cao phan loai
```

```
    report = classification_report(y_test, y_pred, output_dict=True, zero_division=0)
    categories = list(report.keys())[:-3]
    # ngoai tru 'accuracy', 'macro avg', 'weighted avg'

    # tao du lieu ve bieu do
    precisions = [report[cat]['precision'] for cat in categories]
    recalls = [report[cat]['recall'] for cat in categories]
    f1_scores = [report[cat]['f1-score'] for cat in categories]
    model_f1_scores[name] = f1_scores

    # In ra cac thong so
    print(f'\n{name} Metrics:')
    for category in categories:
        print(f"{category}: Precision={report[category]['precision']:.2f},
        Recall={report[category]['recall']:.2f},
        F1-Score={report[category]['f1-score']:.2f}")

    # In ra do chinh xac trung binh
    accuracy = report['accuracy']
    print(f'Overall Accuracy: {accuracy:.2f}\n')

    # Ve bieu do duong
    axs[i // 2, i % 2].plot(categories, precisions, marker='o', label='Precision',
    color='blue')
    axs[i // 2, i % 2].plot(categories, recalls, marker='o', label='Recall',
    color='green')
    axs[i // 2, i % 2].plot(categories, f1_scores, marker='o', label='F1-Score',
    color='red')
    axs[i // 2, i % 2].set_ylim(0, 1)
    axs[i // 2, i % 2].set_title(f'{name} Metrics')
    axs[i // 2, i % 2].set_ylabel('Score')
    axs[i // 2, i % 2].legend()
    axs[i // 2, i % 2].tick_params(axis='x', rotation=45)

    # Ve ma tran confusion matrix
    cm = confusion_matrix(y_test, y_pred, labels=le.classes_)
    sns.heatmap(cm, annot=True, fmt='d', xticklabels=le.classes_,
    yticklabels=le.classes_,
    cmap='Blues', ax=axs_cm[i // 2, i % 2])
    axs_cm[i // 2, i % 2].set_title(f'{name} Confusion Matrix')
    axs_cm[i // 2, i % 2].set_xlabel('Predicted')
    axs_cm[i // 2, i % 2].set_ylabel('True')

# Ve F1 scores cua 3 mo hinh
for name, f1_scores in model_f1_scores.items():
    axs[1, 1].plot(categories, f1_scores, marker='o', label=f'{name} F1-Score')
axs[1, 1].set_ylim(0, 1)
axs[1, 1].set_title('F1 Scores Comparison')
axs[1, 1].set_ylabel('F1 Score')
axs[1, 1].legend()
axs[1, 1].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()
```

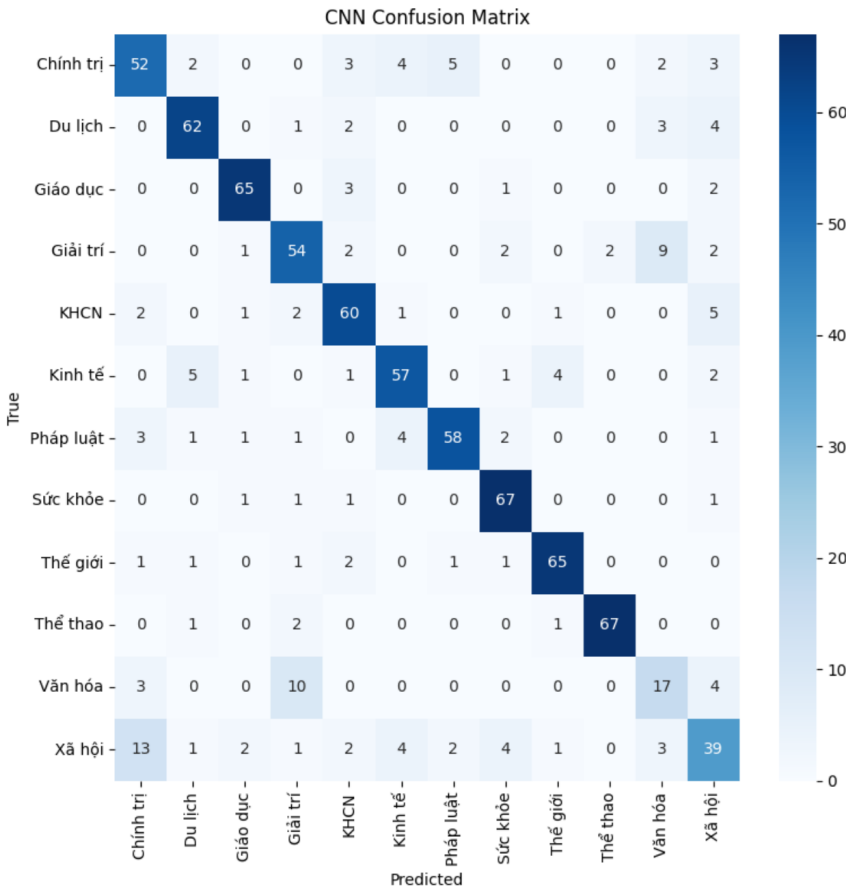
Kết quả sau khi chạy chương trình:

```
Training RNN model...
Epoch [1/10], Loss: 1.2321
Epoch [2/10], Loss: 1.7872
Epoch [3/10], Loss: 0.8874
Epoch [4/10], Loss: 0.2584
Epoch [5/10], Loss: 0.4381
Epoch [6/10], Loss: 0.0719
Epoch [7/10], Loss: 0.0546
Epoch [8/10], Loss: 0.0623
Epoch [9/10], Loss: 0.0302
Epoch [10/10], Loss: 0.0929

RNN Metrics:
Chinh tri: Precision=0.67, Recall=0.72, F1-Score=0.69
Du lich: Precision=0.74, Recall=0.89, F1-Score=0.81
Giao duc: Precision=0.89, Recall=0.87, F1-Score=0.88
Giai tri: Precision=0.77, Recall=0.74, F1-Score=0.75
KHCN: Precision=0.75, Recall=0.79, F1-Score=0.77
Kinh te: Precision=0.85, Recall=0.70, F1-Score=0.77
Phap luat: Precision=0.80, Recall=0.75, F1-Score=0.77
```

```
Suc khoe: Precision=0.90, Recall=0.90, F1-Score=0.90
The gioi: Precision=0.86, Recall=0.89, F1-Score=0.88
The thao: Precision=0.97, Recall=0.90, F1-Score=0.93
Van hoa: Precision=0.54, Recall=0.59, F1-Score=0.56
Xa hoi: Precision=0.59, Recall=0.57, F1-Score=0.58
...
Van hoa: Precision=0.50, Recall=0.50, F1-Score=0.50
Xa hoi: Precision=0.62, Recall=0.54, F1-Score=0.58
Overall Accuracy: 0.81
```

Ma trận confusion matrix của mô hình CNN (Hình 5.12), mô hình RNN (Hình 5.13), LSTM (Hình 5.14):



Hình 5.12: Ma trận confusion matrix của mô hình CNN

5.2.8 Dự đoán trên dữ liệu mới

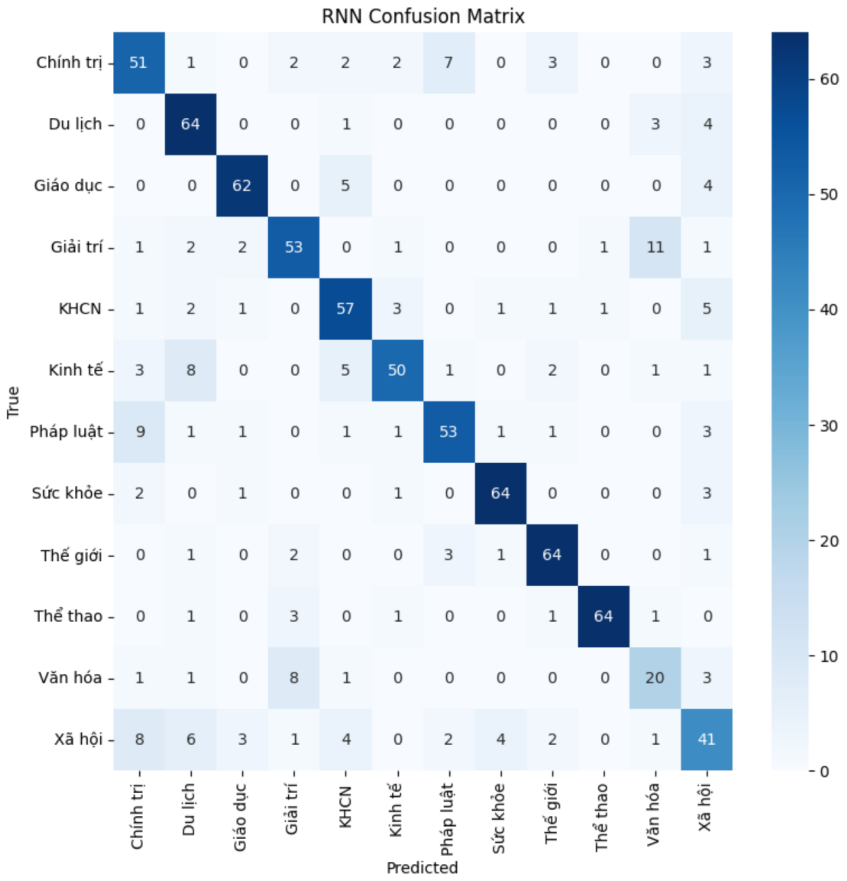
```
new_text = ["sau chien thang cua doi Viet Nam truoc Thai Lan o chung ket luot di ASEAN
cup 2024, cac don vi lu hanh nhanh chong chay tour sang Thai xem tran luot ve]
# Encode the new text using PhoBERT
new_text_phobert = encode_texts(new_text, tokenizer, embeder)

# Move to the correct device
new_text_phobert = new_text_phobert.to(device)

# Predict using the CNN model (assuming it is the best performing model).
deep_models['CNN'].eval()  # Set the model to evaluation mode
with torch.no_grad():
    outputs = deep_models['CNN'](new_text_phobert)
    _, predicted_labels = torch.max(outputs, 1)

predicted_class = le.inverse_transform(predicted_labels.cpu().numpy())
print(f"Predicted class: {predicted_class}")

#For all models:
for model_name, model in deep_models.items():
```



Hình 5.13: Ma trận confusion matrix của mô hình RNN

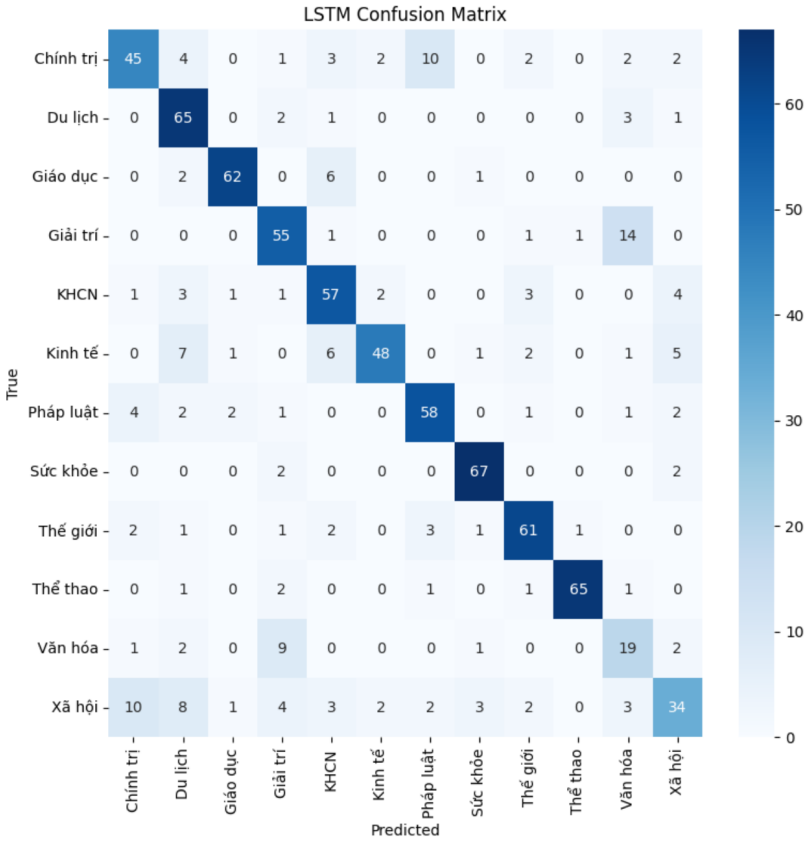
```
model.eval()
with torch.no_grad():
  outputs = model(new_text_phobert)
  _, predicted_labels = torch.max(outputs, 1)
predicted_class = le.inverse_transform(predicted_labels.cpu().numpy())
print(f"Predicted class by {model_name}: {predicted_class}")
```

Kết quả thử nghiệm các mô hình CNN, RNN và LSTM cho bài toán phân loại tin tức được tổng hợp trong bảng 5.2.

Bảng 5.2: Bảng kết quả thử nghiệm trên các mô hình CNN, RNN và LSTM cho bài toán phân loại tin tức tiếng Việt

Model	Accuracy	Precision	Recall	F1-score
CNN	0.81	0.81	0.81	0.81
RNN	0.78	0.79	0.78	0.78
LSTM	0.78	0.78	0.78	0.77

Mặc dù RNN và LSTM có khả năng học đặc trưng tuần tự và phụ thuộc xa, nhưng trong bài toán phân loại tin tức tiếng Việt với tập dữ liệu 4000 văn bản được gán nhãn này, CNN có thể vượt trội nhờ khả năng hiệu quả trong việc nắm bắt các đặc trưng cục bộ (các cụm từ, từ khóa quan trọng), khả năng song song hóa tốt hơn, và có thể là do việc tinh chỉnh kiến trúc phù hợp hơn với đặc điểm của dữ liệu.



Hình 5.14: Ma trận confusion matrix của mô hình LSTM

5.3 Bài tập

Bài 1

Sử dụng code và bộ dữ liệu của bài toán phân loại tin tức ở ví dụ trên, hãy tinh chỉnh siêu tham số mô hình RNN để thăm dò hiệu năng của mô hình.

Bài 2

Sử dụng code và bộ dữ liệu của bài toán phân loại tin tức ở chương 5, hãy tinh chỉnh siêu tham số của mô hình LSTM để thăm dò hiệu năng của mô hình.

Bài 3

Hãy đưa ra mô hình kết hợp LSTM và CNN cho bài toán phân loại cả xúc, thử nghiệm và đánh giá kết quả của mô hình kết hợp.

Bài 4

Mô hình Deep LSTM cho bài toán phân loại tin tức Sử dụng tập dữ liệu mô tả ở mục Hình 5.5 và mô hình Deep LSTM mô tả tại mục 4.1.4, xây dựng mô hình deep LSTM cho bài toán phân loại tin tức.

Bài 5

Mô hình Bi-LSTM cho bài toán phân lớp loại tức Sử dụng tập dữ liệu mô tả ở mục Hình 5.5 và mô hình Bi-LSTM mô tả tại mục 4.1.4, xây dựng mô hình deep LSTM cho bài toán phân loại tin tức.

Bài 6

Sử dụng các kiến thức đã học, thực hiện tinh chỉnh mô hình Transformer.